```
        --recent                     Most Recent
    -s, --size                       Size
```

You can use the method banner to add a message to the top of this help listing as in parser.banner = "Usage: option_1.rb [options]"

And if you call with one of the various options, the appropriate value is set:

```
$ ruby code/rubyworld/option_1.rb -a
"we are sorting by alphabetical"
```

Often, you want a command-line option to take an argument, as in -xSort or --database postgres. You can specify those arguments using OptionParser as well. Generally, the idea is that you add text in all caps after the option, like SORT. The actual text doesn't matter, just that you put some marker there. If the text marker is surrounded by square brackets, the argument is optional, if not, the option is required. There's one slight bit of weirdness here, which is that a required argument is separated from a long option by a space but can be flush against a short option, so -xTHING or -x THING but only --example THING. Optional arguments are separated by a space in both cases. If you are specifying multiple options in the same on call—for example both a long and short option–then the argument only needs to be specified once and it applies to all the options in that call.

In any case, the argument to the option is then passed as an argument to the block:

rubyworld/option_2.rb

```
require "optparse"

parser = OptionParser.new

sort_type = nil

parser.on("-sSORT", "Sort Type") do |value|
  sort_type = value
end

parser.on("-a [DIR]", "Alphabetical") do |value|
  sort_type = :alphabetical
end

parser.on("--recent DATE", "Most Recent") do
  sort_type = :recency
end

parser.on("-h", "--height [METRIC]", "Height") do
  sort_type = :size
end

parser.parse!

p "we are sorting by #{sort_type}"
```

You can do a few more things with options to limit values or coerce types—be sure to check out the official API documentation.

Once you've defined all the options, you need to tell the OptionParser to do something. Again, the OptionParser class provides a few different mechanisms, but most of the time, we think you'll want to call parse!.